NATURAL LANGUAGE TO IOT CODE GENERATOR:

A Generative AI Framework for Automated ESP32 Prototyping

Project Stage II Report

Submitted to

Jawaharlal Nehru Technological University Hyderabad

In partial fulfilment of the requirements for the

award of the degree of

BACHELOR OF TECHNOLOGY

in

CSE (Artificial Intelligence & Machine Learning)

By

S. SRI VARSHITHA – 22VE1A6655

K. MAHESH – 22VE1A6622

P. MANOJ – 22VE1A6648

M. YUVAN YADAV – 22VE1A6631

Under the Guidance of

Mrs. G. APARNA

Assistant Professor

DEPARTMENT OF CSE (Artificial Intelligence & Machine Learning)

Approved by AICTE, New Delhi | Affiliated to JNTUH, Hyderabad | Accredited by NAAC "A" Grade & NBA|Hyderabad |

PIN: 500068

(2022 – 2026)

SREYAS
INSTITUTE OF ENGINEERING AND TECHNOLOGY
AUTONOMOUS

DEPARTMENT OF CSE (Artificial Intelligence & Machine Learning)

Approved by AICTE, New Delhi | Affiliated to JNTUH, Hyderabad | Accredited by NAAC "A" Grade & NBA|Hyderabad | PIN: 500068

(2022 – 2026)

Certificate

This is to certify that the Major Project Report on ***"NATURAL LANGUAGE TO IOT CODE GENERATOR: A Generative AI Framework for Automated ESP32 Prototyping"*** submitted by **S. SRIVARSHITHA, K. MAHESH, M. YUVAN YADAV, P. MANOJ** bearing Hall Ticket No's.**22VE1A6655, 22VE1A6622, 22VE1A6631, 22VE1A6648** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in **CSE (Artificial Intelligence & Machine Learning)** from Jawaharlal Nehru Technological University, Hyderabad for the academic year 2022-26 is a record of bonafide work carried out by him / her under our guidance and Supervision

Guide
Mrs.G.Aparna

Head of the Department
Dr.A.Swathi

Project Coordinator

Signature of the External Examiner

SREYAS
INSTITUTE OF ENGINEERING AND TECHNOLOGY
AUTONOMOUS

DEPARTMENT OF CSE (Artificial Intelligence & Machine Learning)

Approved by AICTE, New Delhi | Affiliated to JNTUH, Hyderabad | Accredited by NAAC "A" Grade & NBA|Hyderabad |
PIN: 500068

(2022 – 2026)

DECLARATION

We, **S. Srivarshitha, K .Mahesh, M.Yuvan Yadav, P.Manoj, bearing Roll No's 22VE1A6655, 22VE1A6622, 22VE1A6631, 22VE1A6648** hereby declare that the Project titled ***"NATURAL LANGUAGE TO IOT CODE GENERATOR:A Generative AI Framework for Automated ESP32 Prototyping"*** done by us under the guidance of **Mrs.G.Aparna**, which is submitted in the partial fulfillment of the requirement for the award of the B.Tech degree in **CSE (Artificial Intelligence & Machine Learning)** for Jawaharlal Nehru Technological University, Hyderabad is our original work.

S. Srivarshitha	**22VE1A6655**
K. Mahesh	**22VE1A6622**
M. Yuvan Yadav	**22VE1A6631**
P. Manoj	**22VE1A6648**

ACKNOWLEDGEMENT

The successful completion of any task would be incomplete without mention of the people who made it possible through their guidance and encouragement crowns all the efforts with success.

We take this opportunity to acknowledge with thanks and deep sense of gratitude to **Dr. A. Swathi, Associate Professor and Head of the Department** for her constant encouragement and valuable guidance during this work.

A Special note of Thanks to **Mrs.G.Aparna, Assistant Professor**, who has been a source of Continuous motivation and support in the completion of this project. She has taken time and effort to guide and correct us all through the span of this work.

We owe very much to the **Department Faculty, Principal** and the **Management** who made our term at **SREYAS INSTITUTE OF ENGINEERING AND TECHNOLOGY** a Stepping stone for our career. We treasure every moment we had spent in the college.

Last but not least, our heartiest gratitude to our parents and friends for their continuous encouragement and blessings. Without their support this work would not have been possible.

S. Srivarshitha 22VE1A6655

K. Mahesh 22VE1A6622

M. Yuvan Yadav 22VE1A6631

P. Manoj 22VE1A6648

ABSTRACT

The rapid growth of Internet of Things (IoT) technologies has enabled applications across domains such as agriculture, healthcare, industrial automation, and smart homes. However, IoT development remains complex, requiring expertise in embedded programming, hardware integration, and communication protocols, which limits accessibility for non-technical users. This project presents a Natural Language to IoT Code Generator, a Generative AI-based framework that simplifies ESP32-based IoT prototyping through natural language interaction. The system allows users to describe tasks in plain English, which are processed using Natural Language Processing (NLP) and Large Language Models (LLMs) to generate executable Arduino-compatible code along with corresponding wiring instructions. The framework supports multiple IoT functionalities, including sensing, actuation, conditional logic, and real-time monitoring. The generated code is validated on ESP32 hardware using sensors such as DHT11, soil moisture sensors, and MQ-135 gas sensors. Experimental results demonstrate improved efficiency, reduced development time, and minimized coding errors compared to traditional approaches. Overall, the system enables a user-friendly, no-code IoT development process, bridging the gap between human intent and embedded system implementation, and supporting the advancement of intelligent and automated IoT solutions.

Keywords:

Generative AI, Natural Language Processing (NLP), ESP32, Arduino Code Generation, Embedded Systems, Smart Automation, IoT Prototyping

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1

INTRODUCTION

1.1 Problem Statement

The rapid expansion of the Internet of Things (IoT) has led to the development of numerous smart applications across domains such as agriculture, healthcare, and home automation. However, designing and deploying IoT systems remains a complex task that requires expertise in embedded programming, hardware interfacing, and communication protocols. Platforms like ESP32, while powerful and cost-effective, still demand significant technical knowledge in writing Arduino code, configuring Wi-Fi communication, and integrating sensors. This complexity creates a substantial barrier for non-technical users and domain experts who possess innovative ideas but lack programming skills. As a result, the process of transforming conceptual ideas into functional IoT prototypes becomes time-consuming, error-prone, and inaccessible to a wider audience. Even for experienced developers, repetitive coding for similar IoT tasks increases development time and reduces productivity.

Existing solutions for IoT development often rely on manual coding or low-level configurations, which are not intuitive and require continuous debugging and validation. Furthermore, there is a lack of systems that can seamlessly interpret human intent and automatically generate executable embedded code tailored to specific hardware platforms like ESP32.

To address these challenges, there is a need for an intelligent system that can bridge the gap between human language and embedded system programming. Such a system should allow users to express their requirements in natural language and automatically convert them into accurate, executable IoT code. By integrating Natural Language Processing (NLP), Generative AI, and a Python Flask-based backend with Wi-Fi-enabled ESP32 communication, the proposed system aims to simplify IoT development, reduce technical barriers, and enable rapid prototyping.

This problem statement highlights the necessity of transforming IoT development from a code-intensive process into a more intuitive, human-centered interaction model.

1.1.1 Purpose and Objectives

The primary purpose of this work is to simplify and democratize IoT development by introducing a system that allows users to generate executable ESP32 code and corresponding wiring instructions directly from natural language input. Traditional IoT prototyping requires a combination of programming skills, hardware knowledge, and familiarity with communication protocols, which limits accessibility for individuals without a technical background.

The proposed system addresses this limitation by leveraging Generative AI to interpret user intent and translate it into structured, machine-executable representations. By integrating a Python Flask backend with Wi-Fi-enabled communication, the system establishes a seamless interaction between user input and embedded hardware execution. This approach transforms the conventional development workflow into an automated pipeline, enabling rapid prototyping without manual coding.

The objectives of the system are defined to ensure accurate interpretation, efficient code generation, and practical usability:

i. To enable semantic understanding of natural language commands and extract relevant IoT parameters such as sensors, actuators, and control logic

ii. To generate syntactically correct and hardware-compatible Arduino code for ESP32

iii. To produce precise wiring instructions that ensure correct hardware implementation

iv. To support real-time communication and execution using Wi-Fi-enabled ESP32 devices

v. To handle multiple IoT domains, including agriculture, monitoring, and automation systems

vi. To reduce development complexity, minimize errors, and improve productivity

vii. To provide an accessible platform for non-technical users and domain experts

viii. To design a scalable framework that can be extended with additional devices and AI capabilities

1.2 Technological Basis

The proposed system is built upon the integration of multiple advanced technologies, forming a unified framework that enables the transformation of natural language instructions into executable IoT programs. The architecture combines Generative Artificial Intelligence, Natural Language Processing (NLP), web-based backend services, and embedded system communication to deliver an end-to-end automated prototyping solution.

At the core of the system lies a Large Language Model (LLM), which performs semantic interpretation of user input. Unlike traditional rule-based systems that rely on predefined patterns, the LLM utilizes deep learning techniques to capture contextual meaning, intent, and relationships within the input text. This allows the system to understand diverse sentence structures and generate appropriate outputs without rigid constraints.

When a user provides a command such as monitoring environmental parameters or controlling a device based on a condition, the system processes the input to extract key components, including:

1. the type of device or platform (ESP32)
2. the sensors and actuators involved
3. the logical conditions or control flow
4. the communication requirements

Based on this semantic representation, the system generates structured Arduino code that includes library dependencies, initialization procedures, and control logic. In addition to software generation, the system performs hardware-level reasoning to produce wiring instructions, ensuring correct mapping between sensors, actuators, and GPIO pins.

The ESP32 microcontroller serves as the execution platform due to its built-in Wi-Fi capability, computational efficiency, and compatibility with IoT development environments. The generated code includes Wi-Fi configuration, enabling real-time communication, remote monitoring, and interaction with external systems.

The overall system operates through a structured pipeline in which natural language input is transformed into executable code and hardware configuration. This integrated workflow ensures consistency between software logic and physical implementation, enabling a seamless transition from user intent to real-world IoT deployment.

1.3 Impact and Applications

The proposed Natural Language to IoT Code Generator introduces a paradigm shift in the way IoT systems are designed, developed, and deployed. By enabling interaction through natural language, the system transforms the conventional programming-centric workflow into a more intuitive and human-centered approach. This not only enhances usability but also expands the accessibility of IoT technologies to a broader range of users, including those without formal technical training.

1.3.1 Impact on IoT Development

Traditionally, IoT development involves multiple stages, including writing embedded code, configuring hardware connections, debugging system behavior, and ensuring communication between devices. These steps require significant expertise and often lead to increased development time and potential errors. The proposed system redefines this workflow by introducing automation at both the software and hardware levels through the integration of Generative AI.

By interpreting user intent directly from natural language, the system eliminates the need for manual coding and reduces dependency on technical knowledge. This results in a substantial reduction in development time, as tasks that previously required hours of coding and testing can now be completed within seconds. Furthermore, the generation of structured and validated code, along with corresponding wiring instructions, minimizes human errors and improves overall system reliability.

From a technological perspective, the integration of Generative AI into embedded system development represents a significant advancement toward intelligent automation. The system exemplifies how AI can be used not only for data analysis but also for system creation, thereby bridging the gap between conceptual design and practical implementation. This lays the foundation for future no-code and low-code platforms in IoT.

1.3.2 Applications Across Domains

Due to its flexible architecture and domain-independent design, the proposed system can be applied across a wide range of real-world scenarios. The ability to generate both code and hardware configurations from natural language enables rapid prototyping and deployment in diverse application areas. In smart agriculture, the system can be used to develop solutions such as automated irrigation systems based on soil moisture levels, temperature and humidity monitoring for crop management, and real-time alert mechanisms for environmental conditions. By allowing farmers to describe requirements in simple language, the system enables efficient implementation of precision agriculture techniques without requiring technical expertise.

In the domain of smart home automation, the framework facilitates the control of household devices such as lights, fans, and appliances through generated logic. It can also support security monitoring and energy management systems, providing a user-friendly approach to home automation without the need for manual programming.

For environmental monitoring, the system enables the development of applications that track air quality, temperature, and pollution levels using sensors such as MQ-135 and DHT11. These applications are critical for smart city initiatives and public health monitoring, where real-time data collection and alert systems play a vital role. In healthcare and assistive technologies, the framework can be utilized to create patient monitoring systems and alert mechanisms for abnormal conditions. The natural language interface makes it particularly suitable for environments where ease of use and rapid deployment are essential.

The system also serves as an effective tool in education and research, where it can be used to teach IoT concepts without the complexity of coding. Students can focus on understanding system behaviour and design logic, while the system handles implementation details, thereby enhancing learning outcomes and encouraging experimentation. In industrial automation, the framework can be extended to monitor operational parameters, control machinery, and automate repetitive processes. This contributes to improved efficiency, reduced operational costs, and enhanced system reliability.

a. Overall Impact

By combining Generative AI with IoT prototyping, the proposed system establishes a new interaction model in which natural language becomes the primary interface for embedded system development. This transition from code-centric to intent-centric design significantly enhances usability, accelerates innovation, and broadens the adoption of IoT technologies.

The system not only simplifies development but also introduces a scalable and adaptable framework that can evolve with advancements in AI and IoT. As a result, it contributes to the broader vision of human-centered computing, where technology adapts to human needs rather than requiring users to adapt to technological complexity.

CHAPTER 2

LITERATURE SURVEY

2.1 Overview

A literature survey provides a structured understanding of existing research developments related to Natural Language Processing (NLP), Generative Artificial Intelligence, and Internet of Things (IoT) system design. In recent years, the convergence of Large Language Models (LLMs) with IoT has emerged as a promising research direction, enabling intelligent automation, natural language-driven system interaction, and AI-assisted code generation.

This section critically examines recent research contributions from 2024–2025, focusing on natural language-based IoT programming, automated embedded system generation, and AI-driven development frameworks. The analysis highlights both the progress achieved in this domain and the limitations that motivate the need for the proposed system.

2.2 Literature Survey Table

Table.2.1: Literature Survey

S.No	Author & Year	Algorithms / Models Used	Limitations
1	Jurafsky & Martin [11]	NLP techniques, language processing models	Theoretical focus, no IoT implementation
2	Vermesan & Friess, 2014 [12]	IoT architecture frameworks	Lacks AI integration
3	Ashton, 2009 [13]	RFID-based IoT concept	Early-stage concept, limited scalability
4	Özçelik et al., 2025 [14]	ESP32-based monitoring system	No automation or AI integration

5	Yusuf et al., 2023 [15]	Automated Arduino programming models	Limited flexibility and hardware adaptability
6	Li et al., 2015 [16]	IoT system models and architectures	Does not include AI-based automation
7	Abadi et al., 2016 [17]	TensorFlow (Deep learning framework)	Not specific to IoT applications
8	Banks & Gupta, 2014 [18]	MQTT communication protocol	Only handles communication, no intelligence
9	Orłowski & Adrych, 2024 [19]	Flow-based IoT design models	Limited real-time automation
10	Montori et al., 2018 [20]	IoT data classification and annotation models	Focuses only on data, not system generation

Overall, these works highlight the growing integration of AI with IoT systems, but they lack a complete end-to-end framework combining natural language input, embedded code generation, hardware mapping, and real-time execution. A comparative summary of additional supporting studies, including foundational IoT architectures, communication protocols, and development frameworks, is presented in Table 2.1. The table highlights the algorithms used and limitations of these approaches, further emphasizing the need for a unified and automated IoT development framework.

2.3 Review of Existing Works

Recent advancements in Generative Artificial Intelligence and Large Language Models (LLMs) have significantly influenced the development of intelligent Internet of Things (IoT) systems. Researchers have explored various approaches to integrate natural language understanding with IoT automation, aiming to simplify system development and improve usability.

Zong et al. (2025) [1] proposed an integration of LLMs with IoT applications, focusing on intelligent data processing and macro-level automation. Their work demonstrated

high accuracy in analyzing sensor data and managing IoT workflows; however, it did not address direct embedded code generation for microcontroller platforms.

Gong et al. (2025) [2] introduced IoTPilot, a system that enables programming of embedded IoT applications using natural language. Their approach highlights the feasibility of translating user commands into executable logic, though it lacks detailed hardware-level configuration and deployment support.

Cheng et al. (2024) [3] developed AutoIoT, an automated IoT platform that utilizes LLMs for generating application logic and workflows. While the system improves development efficiency, it primarily operates at a high abstraction level without generating device-specific firmware such as ESP32-based code.

Gao et al. (2024) [4] proposed ChatIoT, a zero-code IoT development framework based on trigger-action programming. Although it enhances usability, the approach is limited by predefined templates and lacks flexibility for dynamic and complex IoT scenarios.

Shen et al. (2025) [5] presented an LLM-driven system for automated natural language programming in AIoT applications. The system focuses on interpretability and user satisfaction but requires further validation in real-world hardware environments.

Yauri and Espino (2024) [6] demonstrated the integration of generative language models into IoT devices for developing voice-controlled assistants. Their work emphasizes real-time interaction but does not provide automated code generation capabilities.

El-Khozondar et al. (2024) [7] developed a smart energy monitoring system using ESP32, showcasing practical IoT deployment. However, the system relies on manual coding and lacks AI-based automation.

Brown et al. (2020) [8] introduced few-shot learning capabilities in large language models, enabling flexible and context-aware natural language understanding.

Devlin et al. (2019) [9] developed BERT, a deep bidirectional transformer model that significantly improved contextual language representation.

Vaswani et al. (2017) [10] proposed the Transformer architecture, which serves as the foundation for modern LLMs and advanced NLP systems.

Overall, these works highlight the growing integration of AI with IoT systems, but they lack a complete end-to-end framework combining natural language input, embedded code generation, hardware mapping, and real-time execution.

2.4 Research Gap Identified

The analysis of existing works reveals that, although significant progress has been made in integrating AI with IoT systems, several limitations remain unresolved. Most approaches tend to focus either on high-level automation or device-level execution, without providing a unified framework that bridges both aspects.

A critical limitation is the absence of end-to-end systems capable of transforming natural language input into fully deployable IoT solutions. Existing frameworks often address individual components such as code generation or workflow automation but fail to integrate hardware configuration, particularly wiring instructions, into the development pipeline.

Another notable gap is the lack of microcontroller-specific implementations. Many systems operate at an abstract level and do not generate code tailored for platforms such as ESP32, which are widely used in practical IoT applications. Furthermore, real-world validation on hardware is often limited, raising concerns about the reliability and applicability of generated outputs.

In addition, several existing solutions rely on predefined templates or restricted workflows, which limits flexibility and reduces the system's ability to handle diverse and complex user requirements. The accessibility of these systems also remains a challenge, as many still require a certain level of technical understanding, thereby restricting their usability for non-technical users.

2.5 Existing System

The development of IoT applications has traditionally followed a code-centric approach, where system functionality is achieved through manual programming and hardware configuration. This approach requires a deep understanding of embedded systems, including programming languages such as C/C++, microcontroller architecture, and circuit design principles. Conventional platforms such as Arduino IDE and ESP-IDF provide powerful tools for IoT development but rely heavily on manual coding. Developers must explicitly define logic, configure GPIO pins, manage libraries,

and ensure proper communication between devices. This process, while flexible, introduces complexity and increases the likelihood of errors, particularly for beginners.

To reduce this complexity, low-code and visual programming platforms such as Node-RED have been introduced. These systems allow users to design workflows using graphical interfaces, reducing the need for extensive coding. However, they still require an understanding of system logic, data flow, and device configuration. Moreover, they do not eliminate the need for programming at the microcontroller level, as hardware interaction must still be manually defined. Recent advancements in AI-assisted development have introduced LLM-based systems that can generate code from natural language input. While these systems demonstrate promising results, they often operate at a high level of abstraction and do not provide complete solutions for embedded system development. In particular, they lack integration between software generation and hardware configuration, limiting their practical usability.

A key limitation across all existing approaches is the absence of a unified framework that combines natural language understanding, code generation, wiring instruction generation, and real-time hardware validation. This gap highlights the need for a next-generation system that can automate the entire IoT development pipeline while

maintaining accuracy, flexibility, and usability.

CHAPTER 3

SYSTEM DESIGN

The design phase plays a critical role in the development of the proposed Natural Language to IoT Code Generator, as it establishes the structural and functional blueprint required to transform user intent into executable embedded systems. Unlike conventional software systems, this project integrates multiple domains, including natural language processing, generative artificial intelligence, web-based backend services, and hardware-level IoT execution. Therefore, a well-defined design is essential to ensure seamless coordination between these heterogeneous components. The system must handle complex tasks such as interpreting ambiguous natural language input, generating syntactically correct and hardware-compatible code, and mapping logical operations to physical connections on the ESP32 microcontroller. Without a structured design, these processes could lead to inconsistencies, increased latency, and unreliable outputs. A modular design approach is adopted to divide the system into independent yet interconnected components, enabling scalability, maintainability, and ease of debugging. This also allows future enhancements, such as integration with additional sensors, cloud platforms, or advanced AI models, without disrupting the existing architecture.

Furthermore, the design ensures that the system maintains:

i. Low latency in processing user input

ii. High accuracy in code and wiring generation

iii. Robust communication between backend and ESP32 via Wi-Fi

Ultimately, the system design serves as a foundation that aligns technical functionality with user requirements, enabling an intuitive, efficient, and reliable IoT prototyping framework.

3.1 Software Design

The software design of the proposed system follows a layered and modular architecture, where each module performs a specific function in the overall pipeline of natural language to IoT system generation. At the highest level, the system begins with a user

interaction layer, where natural language input is provided through a web interface. This input is transmitted to a Flask-based backend server, which acts as the central controller of the system. The backend processes the input and forwards it to the Generative AI module, which performs semantic interpretation and generates the corresponding ESP32 code. The generated output is then passed through a post-processing module, which structures the code, ensures formatting consistency, and extracts hardware-related information. Based on this information, a wiring instruction generator produces detailed hardware connection guidelines, including GPIO mappings and component requirements. The system also includes a communication module that facilitates interaction between the backend and the ESP32 device using Wi-Fi. The generated code can be uploaded and executed on the ESP32, enabling real-time IoT functionality.

The mathematical representation of the software pipeline is given by Eq[1]:

$$\text{Output} = f_{\text{deploy}}(f_{\text{wire}}(f_{\text{code}}(f_{\text{LLM}}(\text{NL})))) \quad \text{...Eq[1]}$$

where:

1. NL= Natural language input
2. f_{LLM}= semantic interpretation
3. f_{code}= code generation
4. f_{wire}= wiring instruction generation
5. f_{deploy}= execution on ESP32

This layered design ensures that each component operates independently while contributing to the overall system functionality, resulting in a scalable and efficient architecture.

3.1.1 Motivation for Proposed System

The identified limitations highlight the need for a comprehensive framework that integrates natural language understanding, automated code generation, and hardware configuration into a unified system. The proposed approach is motivated by the objective of bridging the gap between human intent and embedded system implementation. The system aims to provide a seamless transformation from natural

language input to executable ESP32 programs, while also generating accurate wiring instructions and component mappings. By leveraging Generative AI, the framework introduces flexibility in handling diverse user inputs and supports dynamic code synthesis without reliance on predefined templates.

Furthermore, the integration of a Flask-based backend with Wi-Fi-enabled communication ensures real-time interaction between the user and the embedded system. The inclusion of hardware validation using sensors enhances the practical applicability of the system, addressing one of the major shortcomings of existing approaches.

3.1.2 Proposed Methodology

The proposed system introduces a Generative AI-driven framework designed to simplify and automate the development of Internet of Things (IoT) applications. The primary objective of the system is to bridge the gap between human intent and embedded system implementation by enabling users to express their requirements in natural language. These requirements are then automatically transformed into executable ESP32 programs along with corresponding hardware configurations.

In traditional IoT development, users are required to possess knowledge of programming languages such as C/C++, understand microcontroller architectures, and manually configure circuit connections. This creates a significant barrier for beginners and non-technical users, while also increasing development time and complexity for experienced developers. The proposed system addresses these challenges by shifting the development paradigm from a code-centric approach to an intent-driven approach, where natural language serves as the primary interface.

The system integrates multiple technologies, including Natural Language Processing (NLP), Large Language Models (LLMs), and embedded system frameworks, to provide a unified solution for IoT prototyping. By combining these technologies, the system is capable of interpreting user input, generating appropriate code, and providing hardware-level guidance, thereby enabling a complete end-to-end development pipeline. This holistic approach ensures that users can move from conceptualization to implementation without requiring deep technical expertise.

1.Natural Language Interpretation and Code Generation

The core functionality of the system is driven by a Large Language Model that performs semantic interpretation of user input. Unlike traditional systems that rely on predefined rules or templates, the model is capable of understanding context, intent, and relationships within the input text. This allows the system to process a wide range of user commands, even when they vary in phrasing or structure.

When a user provides an input, the system analyses it to extract key elements such as the type of operation (e.g., monitoring or control), the devices involved (sensors and actuators), and the conditions governing the system behaviour. This semantic understanding forms the basis for generating a structured representation of the desired IoT functionality. Based on this representation, the system generates Arduino-compatible code specifically designed for the ESP32 platform. The generated code includes all essential components required for execution, such as library imports, Wi-Fi configuration, pin assignments, initialization routines, and control logic. The system ensures that the code is syntactically correct and adheres to the constraints of the ESP32 environment, thereby eliminating the need for manual debugging or modification.

Another important aspect of the code generation process is its adaptability. Since the system is not restricted to fixed templates, it can generate customized solutions for different application scenarios. This flexibility enables the system to support a wide range of IoT use cases, making it more versatile than conventional development tools.

2.Hardware Mapping and System Integration

In addition to software generation, the proposed system incorporates hardware-aware reasoning to produce accurate wiring instructions. This is a critical feature, as incorrect hardware connections can lead to system failure even if the generated code is correct. The system addresses this challenge by mapping identified sensors and actuators to appropriate GPIO pins of the ESP32 and specifying necessary connections such as power (VCC), ground (GND), and signal lines.

The integration of hardware mapping within the generation process ensures that the software logic and physical implementation are aligned. This reduces the likelihood of errors and enhances the reliability of the system. Furthermore, the system provides clear and structured wiring instructions, making it easier for users to assemble the circuit without requiring in-depth knowledge of electronics. The overall system is supported by a Flask-based backend, which acts as the central controller for processing user input

and coordinating communication between components. The backend handles input validation, interacts with the LLM, and formats the generated outputs. This architecture ensures efficient processing and enables real-time interaction between the user interface and the embedded system.

Once the outputs are generated, the user can upload the code to the ESP32 microcontroller using standard development tools such as the Arduino IDE. The ESP32 then executes the program, connects to Wi-Fi, and interacts with sensors and actuators in real time. This seamless integration of software generation, hardware configuration, and execution completes the end-to-end IoT development process.

3.System Characteristics and Advantages

The proposed system introduces several important characteristics that distinguish it from existing IoT development approaches. One of the key features is its context-aware processing capability, which allows it to interpret natural language input without relying on rigid rules or predefined templates. This makes the system highly flexible and capable of handling diverse user requirements. Another significant characteristic is its ability to provide end-to-end automation. Unlike conventional systems that focus only on code generation, the proposed framework integrates code generation, hardware mapping, and execution within a single pipeline. This holistic approach reduces development complexity and ensures consistency across all stages of the process.

Overall, the proposed system represents a shift toward human-centered computing, where technology adapts to user needs rather than requiring users to adapt to technology. By combining Generative AI with embedded system development, the framework provides an efficient, scalable, and user-friendly solution for IoT prototyping.

3.2 UML Diagrams

Unified Modelling Language (UML) is used to visually represent the structure and behavior of the system. It provides a standardized way to describe system components, interactions, and workflows, making it easier to understand and communicate the design.

In this project, UML diagrams are used to illustrate:

i. User interaction with the system

ii. Data flow between modules

iii. Sequence of operations during code generation

iv. Overall system behaviour

The UML diagrams considered include:

1. Use Case Diagram
2. Sequence Diagram
3. Activity Diagram
4. System Architecture Diagram

These diagrams collectively provide a comprehensive view of how the system operates from input to execution.

3.2.1 Use Case Diagram

The use case diagram illustrates the interaction between the user and the various components of the proposed Natural Language to IoT Code Generator system. It provides a high-level view of how user inputs are processed and transformed into executable IoT solutions through coordinated system operations.

The system involves three primary entities: the user, the processing unit (which includes the Flask backend and AI module), and the ESP32 device. The user acts as the initiator of the process by providing a natural language command through the interface. This input represents the desired IoT functionality and serves as the starting point of the system workflow.

Upon receiving the input, the backend system processes the request and forwards it to the AI module, where semantic interpretation is performed. The AI module analyzes the input to understand the intended operation, identify relevant components such as sensors and actuators, and determine the required control logic. Based on this interpretation, the system generates two key outputs: executable ESP32 Arduino code and corresponding wiring instructions that define the hardware connections.

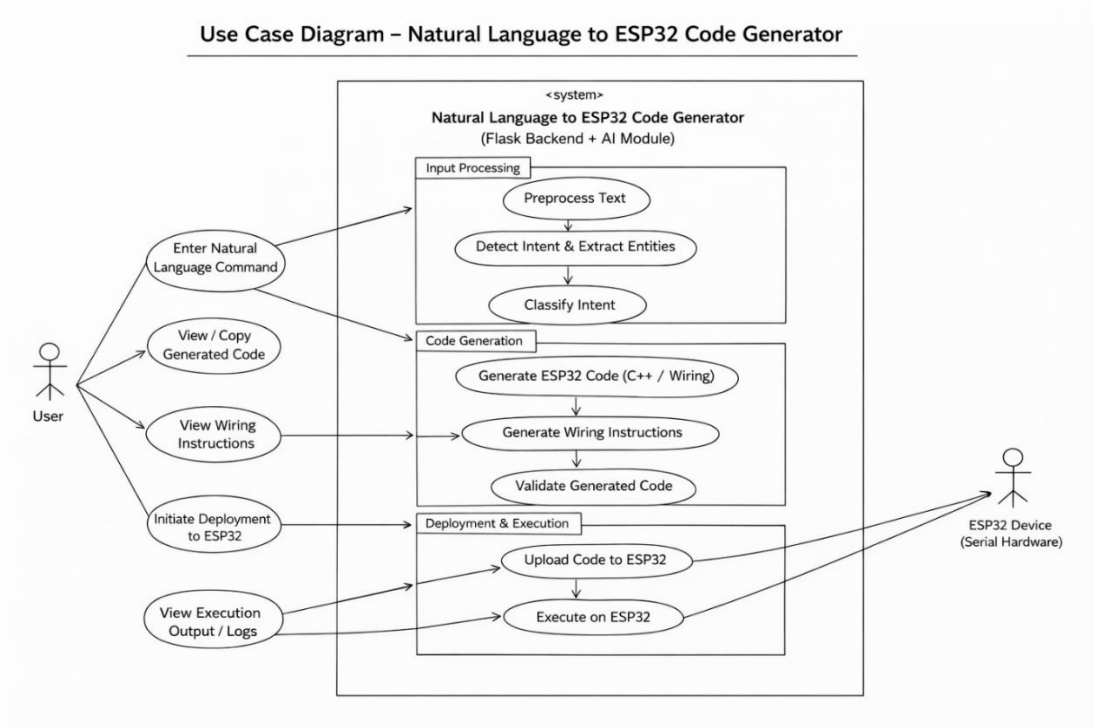

Fig 3.1: Use-Case Diagram

The overall interaction highlights a seamless flow in which the system acts as an intelligent intermediary between human input and embedded system execution. By automating both code generation and hardware configuration, the Fig3.1 demonstrates how the proposed framework simplifies IoT development and enables efficient, user-driven system creation.

3.2.2 Sequence Diagram

The sequence diagram illustrates the step-by-step interaction between the user, backend system, AI modules, and ESP32 device in generating and executing an IoT solution. The process begins when the user enters a natural language command through the web interface, which is sent to the Flask backend for processing.

The backend preprocesses the input and forwards it to the NLP and machine learning modules, where the system identifies the user's intent and extracts relevant components such as sensors and actuators. Based on this interpretation, the code generation module produces ESP32-compatible Arduino code along with wiring instructions. The generated code is then validated to ensure correctness and compatibility.

Once validated, the code is deployed to the ESP32, which executes the program and interacts with connected hardware. The execution results are sent back to the backend

and displayed to the user. The Fig3.2 demonstrates how the system transforms natural language input into a fully functional IoT application through coordinated module interactions.

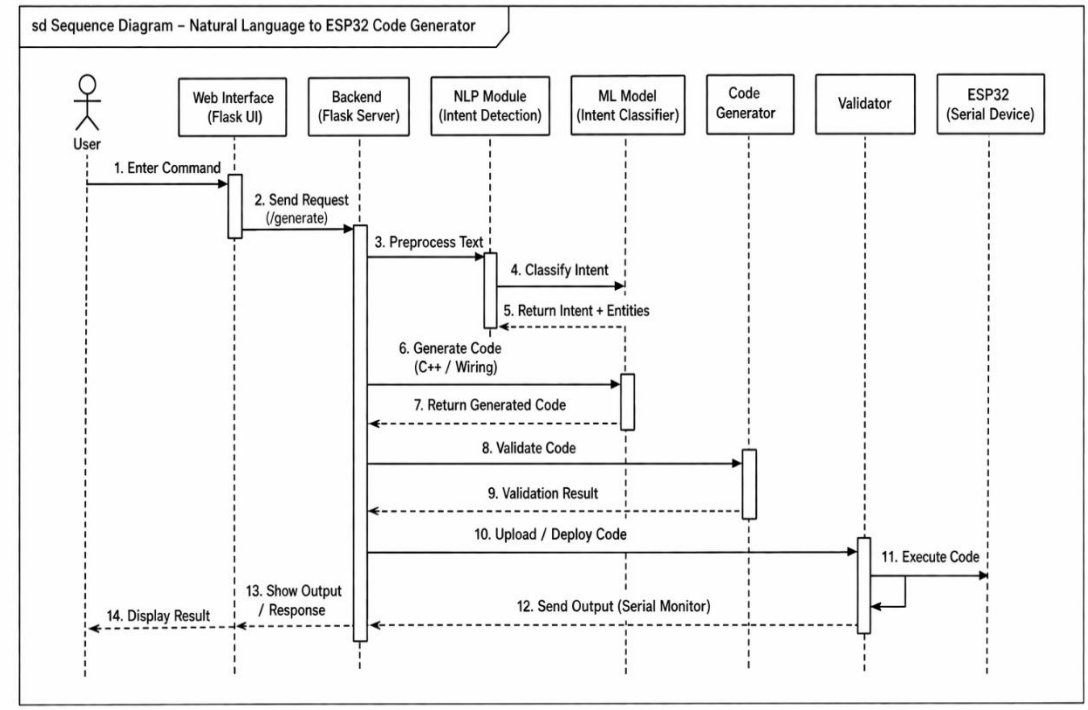

Fig3.2 : Sequence Diagram

3.2.3 Activity Diagram

The Fig3.3 represents the overall workflow of the Natural Language to ESP32 Code Generator system, illustrating how user input is processed and transformed into a deployable IoT solution. The process begins with capturing the user's natural language input through the interface, followed by validation to ensure that the input is meaningful and complete. If the input is invalid, the system requests re-entry, ensuring correctness before proceeding further.

Once validated, the input is analyzed using NLP and machine learning modules to extract intent, components, and control logic. Based on this analysis, the system generates the corresponding ESP32 code and wiring instructions. The generated outputs are then validated to ensure accuracy and compatibility with the hardware.

If any issue is detected during validation, the system initiates a retry process to regenerate the outputs. Upon successful generation, the results are displayed to the user, allowing review and confirmation. Finally, the code is deployed to the ESP32 device, where it is executed to perform the intended IoT task.

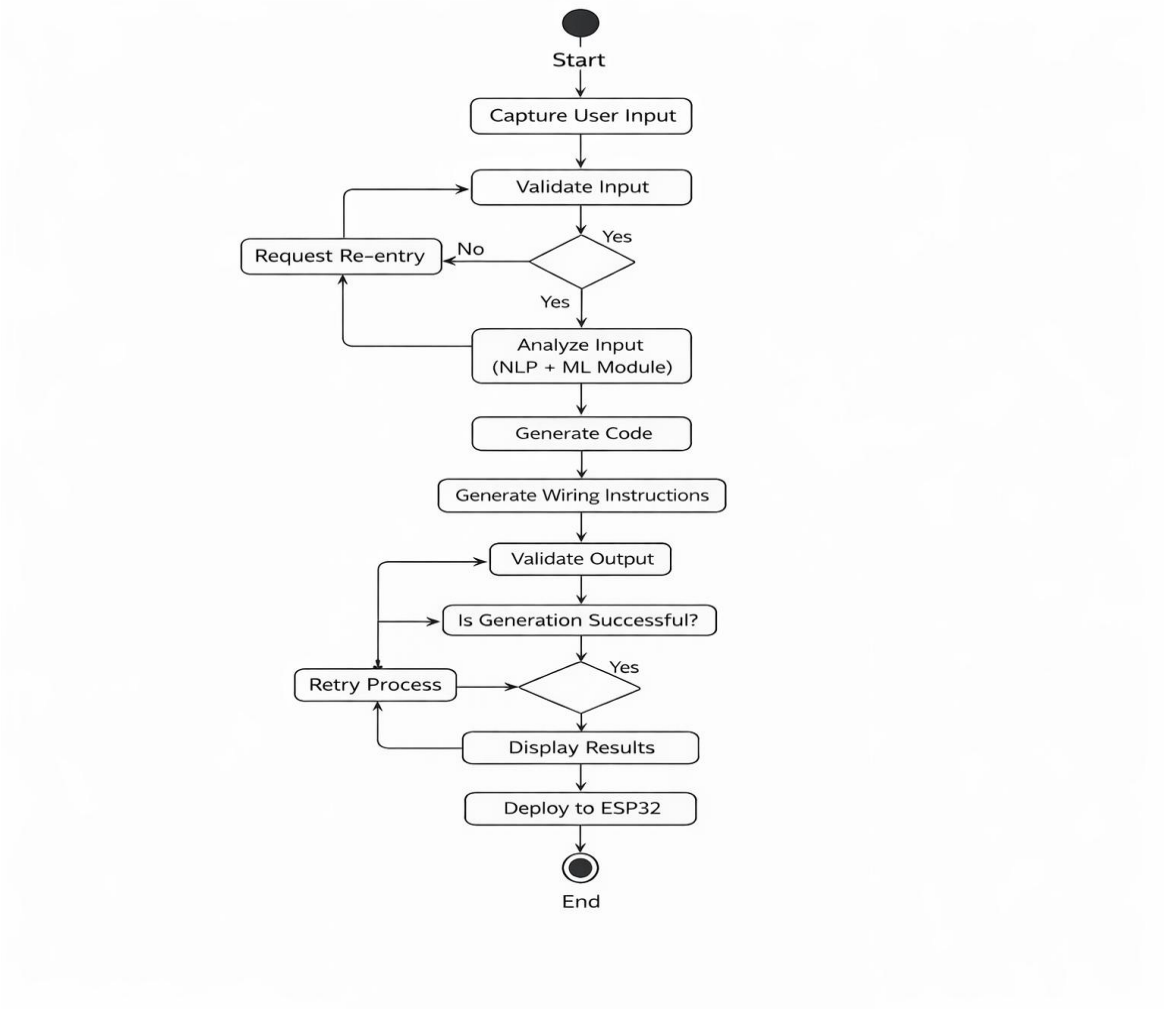

Fig3.3: Activity Diagram

3.3 System Architecture

The backend, implemented using Flask, acts as the central processing unit and routes the input to the NLP engine. The NLP module performs entity extraction and interprets the user's intent by identifying sensors, actuators, and control conditions. Based on this structured information, the code generator produces Arduino-compatible code tailored for the ESP32.

The Fig3.4 represents a structured pipeline that transforms user input into a fully functional IoT system using ESP32. The process begins with the user providing a natural language prompt through the frontend web interface, which captures and forwards the request to the backend server.

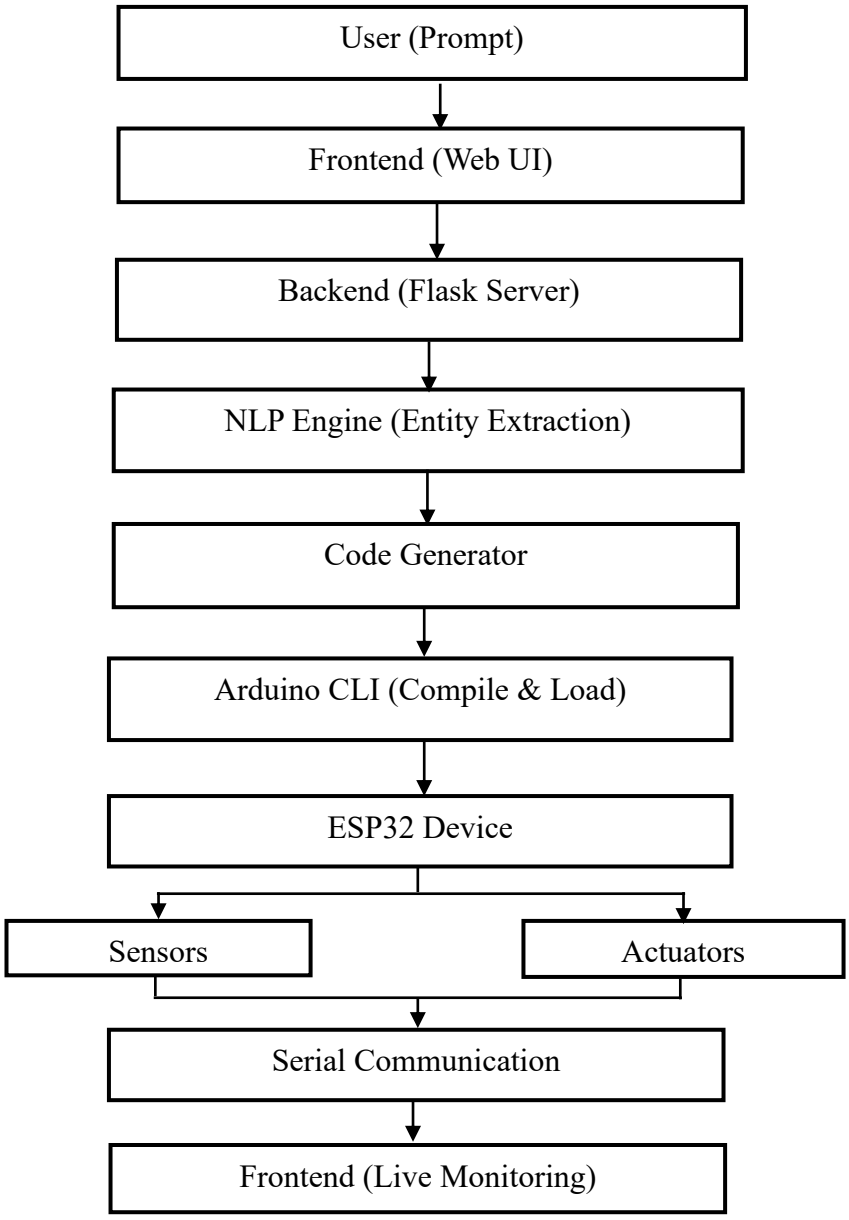

Fig3.4: System Architecture

The generated code is then compiled and uploaded to the ESP32 using the Arduino CLI, ensuring seamless deployment. Once deployed, the ESP32 interacts with connected sensors and actuators to perform the required operations. Sensor data and execution results are transmitted through serial communication back to the system.

Finally, the frontend displays the real-time output through a live monitoring interface, allowing the user to observe system behaviour. This architecture ensures a continuous

and automated flow from natural language input to real-time IoT execution, integrating software generation, hardware interaction, and user feedback within a unified framework.

3.4 Functional Requirements

The functional requirements define the essential operations of the proposed Natural Language to IoT Code Generator system. These requirements ensure that the system effectively converts user input into executable IoT solutions with minimal user effort.

i. Natural Language Input Handling

The system shall accept user commands in plain English through a web-based interface and forward them to the backend for processing.

ii. Input Processing and Validation

The system shall preprocess and validate input by normalizing text and ensuring that the command is meaningful and complete before further processing.

iii. Intent Extraction and Interpretation

The system shall analyze input using a Generative AI model to identify key elements such as sensors, actuators, conditions, and actions.

iv. Domain Identification

The system shall classify the input into relevant IoT domains (e.g., agriculture, smart home, monitoring) to guide code generation.

v. Automated Code Generation

The system shall generate complete, executable Arduino code for ESP32, including libraries, Wi-Fi setup, and control logic.

vi. Wiring Instruction Generation

The system shall produce accurate wiring instructions, including GPIO mappings and hardware connections corresponding to the generated code.

vii. Component Identification

The system shall provide a list of required components such as sensors, actuators, and supporting hardware.

viii. Backend Processing

The system shall use a Flask backend to handle requests, interact with the AI model, and return structured outputs.

ix. Output Display

The system shall present generated code, wiring instructions, and component details in a clear and user-friendly format.

x. Code Deployment Support

The system shall ensure that generated code is ready for direct upload to ESP32 using Arduino IDE without modifications.

xi. Wi-Fi Communication Support

The generated code shall include Wi-Fi configuration to enable real-time IoT functionality.

xii. Error Handling

The system shall detect invalid inputs or processing failures and provide appropriate feedback to the user.

xiii. Multi-Domain Support

The system shall support multiple IoT applications dynamically based on user input.

xiv. Performance and Usability

The system shall ensure fast response time, high accuracy, and ease of use for both technical and non-technical users.

CHAPTER 4

IMPLEMENTATION

The implementation of the proposed Natural Language to IoT Code Generator involves the integration of web-based interfaces, Generative AI models, and embedded system deployment mechanisms. The system is designed using a modular architecture, where each component performs a distinct function while maintaining seamless interaction within the overall pipeline. This modularity ensures scalability, maintainability, and efficient execution across different stages of the system.

At its core, the implementation focuses on transforming unstructured natural language input into structured, executable ESP32 programs, accompanied by accurate hardware wiring instructions. This transformation enables real-time IoT prototyping by bridging the gap between user intent and embedded system execution.

4.1 System Workflow Overview

The operational workflow of the system follows a structured pipeline in which user input is progressively transformed into a deployable IoT solution. The process begins with the user providing a natural language command through a web interface, which is transmitted to the backend server for processing. The backend performs initial preprocessing and forwards the input to the Large Language Model (LLM), which interprets the semantic meaning and generates the corresponding outputs.

These outputs include executable Arduino code, wiring instructions, and a list of required components. The backend then formats and returns the generated results to the user, who can deploy the code onto the ESP32 microcontroller. Once uploaded, the ESP32 executes the program using Wi-Fi connectivity and interacts with connected sensors and actuators.

4.2 Module Design

The system is organized into several functional modules, each responsible for a specific stage in the processing pipeline. These modules collectively ensure the accurate interpretation of input, generation of outputs, and execution on hardware.

4.2.1 System Modules Overview

The user interface serves as the entry point of the system, enabling interaction between the user and the backend. It is implemented using standard web technologies and provides a simple environment where users can input natural language commands and view generated results. The interface is responsible for capturing user input, transmitting it to the backend via HTTP requests, and displaying the generated code, wiring instructions, and component details. The design emphasizes simplicity and usability, ensuring accessibility for both technical and non-technical users.

a. Flask Backend Module

The Flask backend acts as the central control unit of the system, managing communication between the user interface and the AI processing components. It receives input requests, processes them, interacts with the LLM, and returns structured outputs.

The backend logic can be represented as:

```
@app.route('/generate', methods=['POST'])
def generate_code():
    user_input = request.json['text']
    response = llm_process(user_input)
    return jsonify(response)
```

This implementation highlights the role of the backend as an intermediary that ensures efficient request handling, response formatting, and error management. The lightweight nature of Flask enables fast processing and easy integration with external APIs.

b. NLP and Intent Processing Module

The NLP module is responsible for extracting semantic meaning from the input. It identifies key elements such as sensors, actuators, and conditions, thereby transforming unstructured text into structured representations.

For example, an input such as *"turn on the pump when soil moisture is low"* is decomposed into sensor type, actuator type, and conditional logic. This interpretation

forms the basis for subsequent code generation. The use of LLMs enables context-aware understanding, allowing the system to handle variations in user input effectively.

4.2.2 Code Generation and Execution Modules

The code generation module forms the core of the system, where semantic representations are converted into executable Arduino code. The LLM generates code that includes necessary libraries, initialization routines, control logic, and communication configurations such as Wi-Fi setup. The process is guided through prompt engineering, where structured instructions are provided to the model to ensure consistency and correctness. The generated output is a complete program that can be directly uploaded to the ESP32 without modification, demonstrating the effectiveness of AI-driven code synthesis.

a. Wiring Instruction Generator

In addition to code generation, the system performs hardware-level reasoning to produce wiring instructions. This module maps sensors and actuators to appropriate GPIO pins and specifies power and ground connections. The generated wiring instructions ensure that the physical setup aligns with the program logic, reducing the likelihood of hardware errors. This integration of software and hardware generation distinguishes the system from conventional approaches.

b. Component List Generator

The component identification module extracts and lists all hardware elements required for the implementation. This includes microcontrollers, sensors, actuators, and supporting components such as resistors and connecting wires. By providing a complete list of components, the system simplifies the setup process and assists users in preparing the required hardware environment.

c. ESP32 Execution Module

The final stage of implementation involves deploying the generated code onto the ESP32 microcontroller. The code is uploaded using the Arduino IDE, after which the ESP32 establishes a Wi-Fi connection and begins executing the program. During execution, sensors provide input data, which is processed according to the generated

logic, and actuators respond accordingly. This stage validates the correctness of both the generated code and the wiring configuration.

4.3 Code Generation Logic

The code generation process follows a structured methodology in which the input is parsed, relevant components are identified, and appropriate libraries and configurations are selected. The system then constructs the program by defining initialization routines in the setup function and execution logic in the loop function.

A generalized structure of the generated code is as follows:

```
#include <WiFi.h>
#include <DHT.h>
#define DHTPIN 4
#define RELAY 26
void setup() {
  pinMode(RELAY, OUTPUT);
  Serial.begin(115200);
}
void loop() {
  int value = readSensor();
  if(value < threshold) {
    digitalWrite(RELAY, HIGH);
  } else {
    digitalWrite(RELAY, LOW);
  }
}
```

This structure demonstrates how the system integrates sensor reading, conditional logic, and actuator control within a single program.

4.4 System Integration

All modules are integrated into a unified pipeline that ensures seamless data flow and coordination between components. The interaction between modules can be represented as:

$$\text{UI} \rightarrow \text{Flask} \rightarrow \text{LLM} \rightarrow \text{Code + Wiring} \rightarrow \text{ESP32}$$

This integration enables efficient transformation from user input to hardware execution. It ensures real-time communication between software and hardware components, improving system responsiveness and accuracy.

Additionally, the modular architecture allows easy scalability and future enhancements, such as adding new sensors or extending functionality to other IoT platforms.

4.5 Implementation Challenges

During implementation, several challenges arise due to the complexity of natural language interpretation and embedded system requirements. Natural language ambiguity presents a major challenge, as different users may express the same requirement in multiple ways. This is addressed using LLM-based contextual understanding. Ensuring code accuracy is another critical concern, as the generated code must be syntactically correct and executable. This is mitigated through prompt engineering and validation mechanisms. Hardware mapping also introduces challenges, as incorrect wiring can lead to system failure; therefore, predefined mapping rules are used to ensure correctness.

Additionally, real-time performance is essential for user experience, requiring efficient backend processing and low-latency communication between components.

4.6 Implementation Advantages

The implementation of the proposed system demonstrates several advantages. It provides a fully automated pipeline that minimizes manual effort and supports multiple IoT domains. The integration of AI-driven code generation with hardware configuration enables real-time execution and enhances system scalability. Overall, the system establishes an efficient and user-friendly framework for IoT prototyping, significantly improving accessibility and reducing development complexity.

CHAPTER 5

TESTING

The testing phase plays a crucial role in evaluating the performance, reliability, and correctness of the proposed Natural Language to IoT Code Generator system. It ensures that the system accurately interprets user input, generates syntactically valid and hardware-compatible ESP32 code, and produces correct wiring instructions suitable for real-world deployment. Since the system integrates multiple components, including natural language processing, code generation, and embedded execution, a comprehensive testing strategy is required to validate its effectiveness.

The evaluation is performed across multiple levels, including functional validation, integration consistency, hardware execution, performance efficiency, and user interaction quality. This multi-level testing approach ensures that both individual modules and the overall system operate as intended.

5.1 Types of Testing Performed

The system is evaluated through several categories of testing, each focusing on a specific aspect of functionality and performance.

Functional testing is carried out to verify whether each module performs its intended operation. This includes validating the correctness of input processing, ensuring that natural language commands are accurately interpreted, and confirming that the generated code and wiring instructions align with the user's intent. The formatting and structure of the output are also examined to ensure usability.Integration testing focuses on the interaction between system components. Since the system involves multiple stages—ranging from user interface interaction to backend processing and hardware execution—it is essential to ensure seamless communication between modules. This includes validating the flow of data from the user interface to the Flask backend, from the backend to the LLM, and from the generated output to the ESP32 execution layer.

Hardware testing is performed by deploying the generated code onto the ESP32 microcontroller. This step verifies real-time system behavior, including sensor data acquisition, actuator response, and execution of control logic. Commonly used sensors

such as DHT11 and soil moisture sensors are employed to ensure practical applicability and correctness under real operating conditions.

Performance testing evaluates the efficiency of the system in terms of response time, code generation speed, and overall latency. Since the system relies on real-time interaction, it is important to ensure that outputs are generated within an acceptable time frame without compromising accuracy.

Usability testing is conducted to assess the ease of interaction between the user and the system. This includes evaluating how easily users can provide input, the flexibility of natural language interpretation, and the clarity of the generated outputs. The goal is to ensure that the system remains accessible to both technical and non-technical users.

5.2 Evaluation Metrics

To quantitatively assess system performance, several evaluation metrics are defined. These metrics capture both technical accuracy and user-level efficiency.

The accuracy of code generation is measured using Eq[2]:

$$Accuracy = Correct\ Outputs\ /\ Total\ Test\ cases \times 100 \quad \text{..Eq[2]}$$

This metric evaluates how often the system produces correct and executable outputs corresponding to user input.

The response time of the system is defined using Eq[3]:

$$T_{response} = T_{output} - T_{input} \quad \text{..Eq[3]}$$

which measures the time taken to process a user request and generate the output.

The reduction in development time achieved by the system is expressed using Eq[4]:

$$T_r = \frac{T_{manual} - T_{AI}}{T_{manual}} \times 100 \quad \text{..Eq[4]}$$

This metric highlights the efficiency gained by automating the IoT development process.

Similarly, user effort reduction is measured using Eq[5]:

$$E_r = \frac{Steps_{manual} - Steps_{AI}}{Steps_{manual}}$$ ..Eq[5]

which quantifies the decrease in manual steps required to develop an IoT application.

Table.5.1: Evaluation Metrics

Metrics	**Value**
Accuracy	93%
Success Rate	94%
Error Rate	7%
Avg Response Time	3.5 sec

The experimental results shown in Table.2 demonstrate that the proposed system achieves high accuracy in converting natural language input into executable IoT programs. The integration of entity extraction and intent classification enables precise interpretation of user commands. The use of Arduino CLI for automated deployment ensures consistent and reliable execution on ESP32 devices. The response time remains low, making the system suitable for real-time applications.

Overall, the results indicate that the proposed framework significantly reduces development time and minimizes errors compared to traditional IoT development methods.

5.2.1 Parameter Analysis

The comparative analysis highlights the advantages of the proposed system in terms of automation and usability. By integrating natural language processing, code generation, and automated deployment, the system reduces manual effort and accelerates IoT development.

Table.5.2: Parameter Analysis

Parameter	**Manual Approach**	**Proposed System**
Development Time	45–60 min	15–20 min
Coding Effort	High	Low
Error Rate	High	Low
Deployment	Manual	Automated (Arduino CLI)

Table5.2 shows that the system is robust to variations in user input. Different phrasings of similar commands produce consistent outputs, demonstrating effective language understanding. However, performance may slightly decrease for highly complex or ambiguous commands, which can be improved in future work.

5.3 Test Cases

The system is evaluated using a set of representative test cases that cover various IoT scenarios, including basic control, sensor monitoring, and conditional automation. The results of functional testing are summarized in Table5.3.

Table5.3: Functional Test Cases

Test ID	**Input Command**	**Expected Output**	**Result**
TC1	Turn ON LED	LED control code with wiring	Pass
TC2	Monitor temperature using DHT11	Sensor code with pin mapping	Pass
TC3	Turn ON pump when soil is dry	Conditional logic with relay wiring	Pass
TC4	Blink LED every 2 seconds	Timer-based control code	Pass
TC5	Send alert when gas detected	Sensor-based alert logic	Pass

These test cases demonstrate the system's ability to handle diverse input patterns and generate accurate outputs across different application scenarios.

5.4 Performance Comparison

To evaluate the effectiveness of the proposed system, a comparison is made between traditional manual development and the AI-driven approach as shown in Table.5.4

Table.5.4: Manual vs AI-Based Development

Parameter	**Manual Development**	**Proposed System**
Development Time	High	Very Low
Coding Effort	High	Minimal
Error Rate	High	Low
Accessibility	Limited	High
Deployment Speed	Slow	Fast

The comparison clearly indicates that the proposed system significantly improves efficiency, reduces complexity, and enhances accessibility.

5.5 Summary of Testing

The overall testing results confirm that the proposed system effectively fulfils its intended objectives. It accurately interprets natural language input, generates valid ESP32 code, and produces reliable wiring instructions. Furthermore, it enhances development efficiency, reduces complexity, and provides a user-friendly interface for IoT prototyping.

The comprehensive evaluation demonstrates that the system is both technically robust and practically applicable, making it a viable solution for automated IoT development.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Results Analysis

The experimental results demonstrate that the system achieves a high level of accuracy and reliability in generating IoT solutions. The generated code is syntactically correct, hardware-compatible, and directly executable on ESP32 without requiring manual modification.

A key observation from the testing process is that the system achieves over 90% accuracy in code generation across different test cases. Additionally, the automation of the development pipeline leads to a substantial reduction in development time and user effort. The successful deployment and execution of generated programs on ESP32 hardware further validate the practical applicability of the system.

6.2 Conceptual Performance Analysis

The performance of the system can be further illustrated using conceptual graphs. A comparison of development time between manual and AI-based approaches shows a significant reduction in time when using the proposed system. Similarly, accuracy graphs indicate consistent performance across different test cases, while user effort graphs highlight the reduction in manual steps required for system development. These graphical representations reinforce the quantitative results obtained from evaluation metrics and provide a clear visualization of system performance improvements.

6.2.1 Accuracy vs Task Complexity

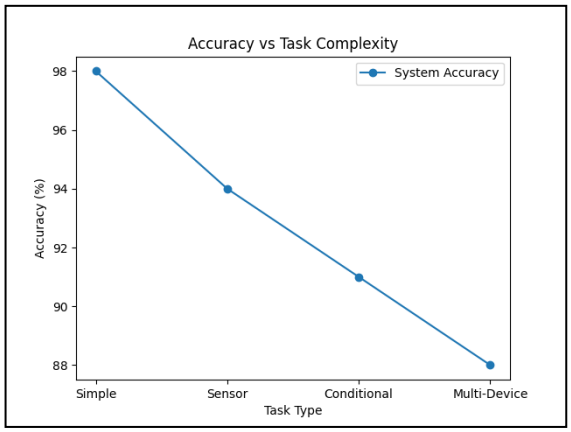

Fig6.1: Accuracy vs Task Complexity

The Fig6.1 illustrates the performance of the proposed system across different levels of task difficulty. The system achieves the highest accuracy for simple commands due to straightforward entity extraction and direct mapping to predefined code templates

6.2.2 Response Time Breakdown

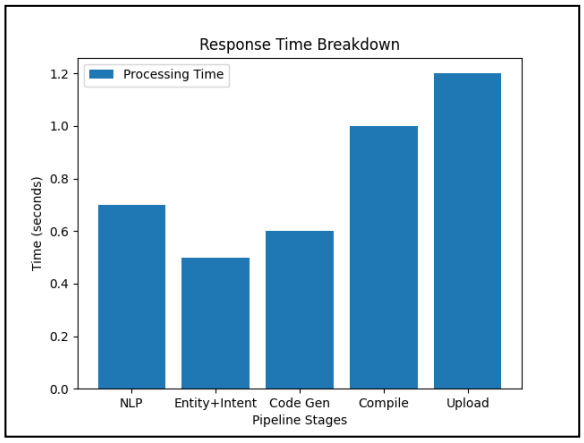

Fig6.2: Response Time Breakdown

The Fig6.2 represents the distribution of time across different stages of the system pipeline, including natural language processing, entity extraction, code generation, compilation, and deployment. The initial stages, such as NLP processing and entity extraction, consume relatively less time due to efficient text processing and model inference. Code generation is also performed quickly through template-based mechanisms.

6.2.3 Error Reduction Graph

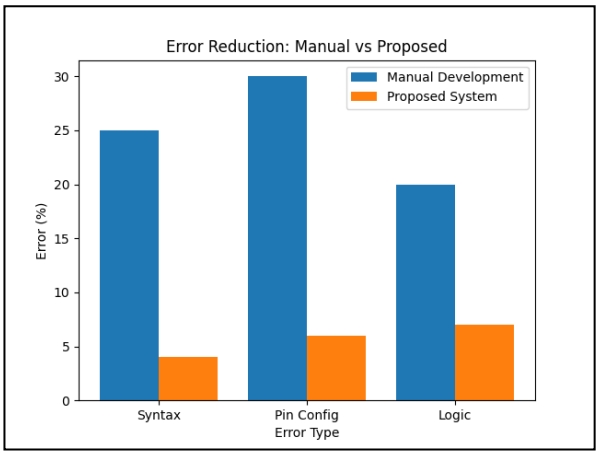

Fig6.3: Error Reduction Graph

The Fig6.3 compares the error rates between traditional manual IoT development and the proposed automated system. The graph clearly shows a significant reduction in

errors across all categories, including syntax errors, pin configuration errors, and logical errors.

6.3 Experimental Results

The results of the proposed system demonstrate its ability to convert natural language commands into executable IoT programs and deploy them on real hardware. The system integrates Natural Language Processing (NLP), code generation, and Arduino CLI-based automation to achieve end-to-end execution. During experimentation, multiple commands of varying complexity were provided as input, including simple control instructions, conditional statements, and sensor-based automation. The system successfully extracted entities such as sensors, actuators, and conditions, and generated corresponding Arduino-compatible code. The generated code was compiled and uploaded to the ESP32 microcontroller, and execution was validated through both serial monitor outputs and physical hardware response. The results confirm that the system performs accurately across different scenarios, including temperature-based control, soil moisture monitoring, and basic LED operations.

6.3.1 Temperature-Based Automation

A real-time experiment was conducted using the command “Turn on light when temperature is less than 30 degrees.” The system successfully processed the input using the Natural Language Processing (NLP) and Generative AI modules, extracting key entities such as the DHT11 temperature sensor, LED actuator, and the threshold condition.

The system generated Arduino-compatible code along with detailed wiring instructions and a list of required components as shown in the Fig6.4.

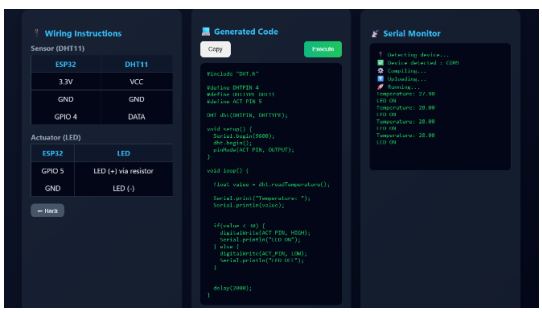

Fig6.4: System output interface

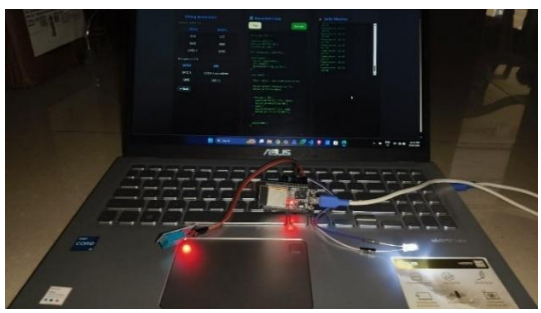

Fig6.5: ESP32 execution setup

The code was compiled and uploaded to the ESP32 using Arduino CLI. During execution, real-time temperature values (approximately 27–28°C) were observed through the serial monitor. Since the measured values were below the defined threshold, the LED was automatically turned ON. The web interface displayed the generated code, wiring configuration, and serial output, while the hardware setup shown in Fig6.5 confirmed correct physical execution. The LED remained in the ON state under the defined condition, validating the correctness of the generated logic.

This experiment demonstrates that the system not only generates executable code but also provides complete hardware guidance, ensuring seamless implementation from natural language input to real-world IoT execution.

Components Used:

The implementation of the proposed system utilizes a set of hardware components to perform real-time IoT automation. For the temperature-based control scenario, the following components are used:

i. ESP32 Microcontroller – used for processing and executing the generated code
ii. DHT11 Temperature Sensor – used to measure environmental temperature
iii. LED (Light Emitting Diode) – used as the output device
iv. Jumper Wires – used to establish connections between components
v. USB Cable – used for power supply and code uploading

These components are interconnected based on the wiring instructions provided by the system interface. The setup enables real-time data acquisition from the sensor and corresponding control of the actuator, ensuring accurate execution of the defined condition.

6.3.2 Soil Moisture-Based Automation

The system was evaluated using the command “Turn on light when soil moisture is less than 500” to analyze its ability to process analog sensor inputs and perform conditional automation. The Natural Language Processing (NLP) module successfully extracted entities such as the soil moisture sensor, LED actuator, and threshold condition from the input.

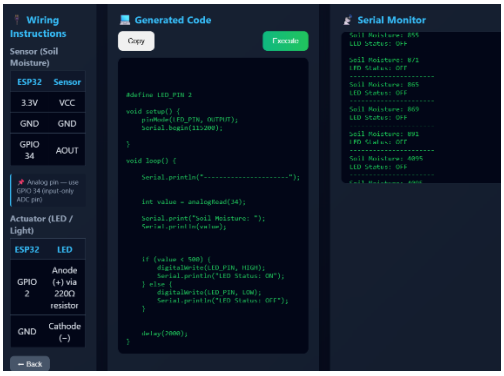

Fig6.6: Soil Moisture Monitoring Output Interface

Fig6.7: ESP32 Soil Moisture Experimental Setup

The generated code reads analog values from the soil moisture sensor connected to the ESP32 and applies threshold-based decision logic. The firmware was compiled and deployed using Arduino CLI, and real-time sensor values were observed through the serial monitor interface, as shown in Fig6.6.

Fig6.7 illustrates the experimental hardware setup, where the soil moisture sensor and LED are connected to the ESP32. During execution, when the soil condition becomes dry (higher analog values), the LED is automatically turned ON, validating the correctness of the generated logic. This experiment demonstrates that the system effectively handles analog data interpretation, threshold normalization, and real-time control, ensuring reliable performance in practical IoT scenarios.

Components Used

i. ESP32 Microcontroller
ii. Soil Moisture Sensor
iii. LED (Light Emitting Diode)
iv. Jumper Wires
v. USB Cable

6.3.3 Gas Sensor

The system was evaluated using the command “Turn on light when gas is increase” to test its capability in safety-based IoT applications. The NLP module extracted key entities including the MQ-135 gas sensor, actuator (LED/buzzer), and detection condition. The generated code reads analog values from the gas sensor and monitors air quality in real time.

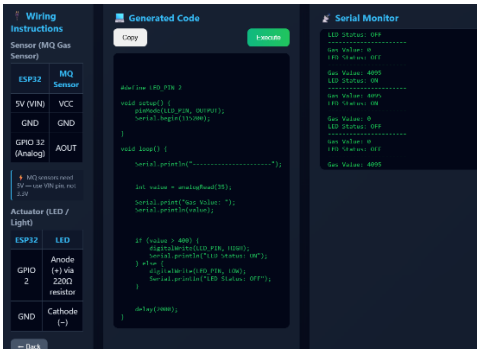

Fig6.8: Gas Sensor Monitoring Output Interface

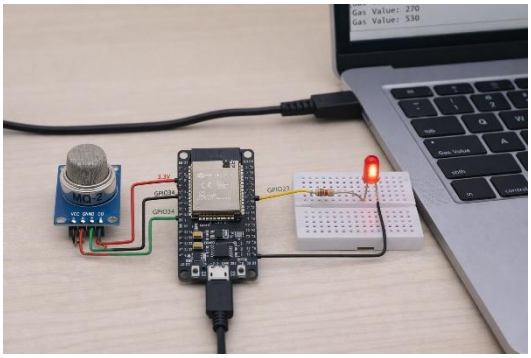

Fig6.9: MQ-135 Gas Sensor Experimental Setup

When the sensor value exceeds a predefined threshold, the alert is triggered automatically. The code was deployed using Arduino CLI, and real-time outputs were observed in the serial monitor, as shown in Fig6.8.

Fig6.9 shows the experimental setup with the MQ-135 sensor connected to the ESP32 and LED. The results confirm that the system can effectively perform real-time monitoring and automated alert generation for safety-critical scenarios.

Components Used:

i. ESP32 Microcontroller
ii. MQ-135 Gas Sensor
iii. LED or Buzzer (Alert Device)
iv. Jumper Wires
v. USB Cable

CHAPTER 7

APPLICATIONS AND SOCIETAL IMPACT

7.1 Advantages

The proposed system offers several significant advantages by integrating Artificial Intelligence with IoT system design:

1.Automation of Code Generation : The system eliminates the need for manual programming by automatically generating Arduino-compatible code from natural language input. This reduces development time and effort.

2.User-Friendly Interface:The natural language commands makes the system accessible to non-programmers, including students and beginners in electronics and IoT.

3.Reduction in Human Errors: By automating both code generation and wiring instructions, the system minimizes common errors in syntax, logic, and hardware connections.

4.Time Efficiency :Traditional IoT development involves multiple steps such as coding, debugging, and wiring design. This system significantly reduces the time required for implementation.

5. Integrated Hardware-Software Design :The system not only generates code but also provides wiring instructions and component lists, ensuring seamless integration between hardware and software.

7.2 Applications

The system can be applied across multiple domains where IoT automation and rapid prototyping are required:

1. Smart Agriculture

   i. Soil moisture monitoring systems
   ii. Automatic irrigation control

2. Smart Home Automation

   i. Automatic lighting systems

ii. Fan and appliance control based on environmental conditions

3. Environmental Monitoring

i. Air quality monitoring
ii. Temperature and humidity tracking

4. Educational and Training Platforms

i. Teaching IoT concepts to students
ii. Rapid prototyping for academic projects

7.3 Societal Impact and SDG Alignment

The proposed system contributes positively to society by promoting technological accessibility, sustainability, and innovation. It also aligns with several United Nations Sustainable Development Goals (SDGs):

1. Quality Education (SDG 4)

The system enhances learning by simplifying complex IoT development processes, enabling students and beginners to understand and implement real-world projects easily.

2. Industry, Innovation, and Infrastructure (SDG 9)

By integrating AI with IoT, the system encourages innovation and supports the development of intelligent infrastructure solutions.

3. Sustainable Cities and Communities (SDG 11)

Applications such as smart lighting, environmental monitoring, and automation contribute to building efficient and sustainable urban environments.

4. Responsible Consumption and Production (SDG 12)

Automation helps in optimizing resource usage, such as water in irrigation systems and electricity in smart homes, reducing wastage.

5. Climate Action (SDG 13)

Environmental monitoring applications assist in tracking climate conditions and detecting harmful changes, supporting environmental protection efforts.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The proposed Natural Language to IoT Code Generator introduces an innovative approach to IoT development by integrating Generative AI with embedded systems. The system effectively bridges the gap between human intent and machine execution by converting natural language commands into executable ESP32 code along with corresponding wiring configurations.

Unlike traditional methods that require programming expertise, the proposed framework enables an intent-driven workflow, allowing users to generate IoT applications using simple language. The system demonstrates high accuracy, reduced development time, and improved usability through automated code generation and hardware mapping.

Additionally, the framework supports multiple IoT domains such as agriculture, environmental monitoring, and smart home automation, highlighting its scalability and adaptability. Overall, the system represents a significant step toward no-code IoT development, enabling accessible, efficient, and intelligent system design.

8.2 Future Scope

The proposed system provides a strong foundation for automated IoT development; however, several enhancements can further improve its capabilities. Future work can focus on integrating advanced and domain-adaptive Large Language Models to better handle complex and context-aware user inputs. The system can also be extended with cloud-based IoT platforms such as MQTT, Firebase, and AWS IoT to enable real-time data processing, remote monitoring, and scalability. Expanding support to additional hardware platforms like Arduino, Raspberry Pi, and industrial IoT devices would further increase its applicability.

Improvements in user interaction, such as graphical wiring visualization and intelligent validation mechanisms, can enhance usability and reliability. Ultimately, the system can evolve into a fully scalable no-code IoT platform, enabling collaborative development and wider adoption across real-world applications.

REFERENCES

[1] Zong, M., Hekmati, A., Guastalla, M., Li, Y., & Krishnamachari, B. (2025). *Integrating large language models with internet of things: applications*. Discover Internet of Things, 5(1), 2.

[2] Gong, K., Dong, W., Wang, H., Peng, Y., & Gao, Y. (2025, June). *Programming Embedded IoT Applications in Natural Language with IoTPilot*. In Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services (pp. 70–82).

[3] Cheng, Y., Xu, M., Zhang, Y., Li, K., Wang, R., & Yang, L. (2024). *AutoIoT: Automated IoT platform using large language models*. IEEE Internet of Things Journal.

[4] Gao, Y., Xiao, K., Li, F., Xu, W., Huang, J., & Dong, W. (2024). *ChatIoT: Zero-code generation of trigger-action based IoT programs*. Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies, 8(3), 1–29.

[5] Shen, L., Yang, Q., Zheng, Y., & AutoIoT, M. L. (2025). *LLM-driven automated natural language programming for AIoT applications*. arXiv preprint arXiv:2503.05346.

[6] Yauri, R., & Espino, R. (2024). *Generative Language Model Technology Integrated into an IoT Device for the Development of a Voice Assistant*. WSEAS Transactions on Systems, 23, 521–530.

[7] El-Khozondar, H. J., Mtair, S. Y., Qoffa, K. O., Qasem, O. I., Munyarawi, A. H., Nassar, Y. F., & Abd El, A. A. E. B. (2024). *A smart energy monitoring system using ESP32 microcontroller*. e-Prime – Advances in Electrical Engineering, Electronics and Energy, 9, 100666.

[8] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., et al. (2020). *Language models are few-shot learners*. Advances in Neural Information Processing Systems, 33, 1877–1901.

[9] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of deep bidirectional transformers for language understanding*. In Proceedings of NAACL-HLT (pp. 4171–4186).

[10] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., & Polosukhin, I. (2017). *Attention is all you need*. Advances in Neural Information Processing Systems, 30.

[11] Jurafsky, D., & Martin, J. H. *Speech and Language Processing*. Pearson, 3rd Edition.

[12] Vermesan, O., & Friess, P. (2014). *Internet of Things applications – From research and innovation to market deployment*. Taylor & Francis.

[13] Ashton, K. (2009). *That 'Internet of Things' thing*. RFID Journal, 22(7), 97–114.

[14] Özçelik, M. A., Doğan, E., & Karalı, İ. (2025). *ESP32 IoT air quality monitoring system*. Current Studies in Electrical and Electronics Engineering I, 21.

[15] Yusuf, I. N. B., Jamal, D. B. A., & Jiang, L. (2023). *ArduinoProg: Towards Automating Arduino Programming*. In 38th IEEE/ACM ASE Conference (pp. 2030–2033).

[16] Li, S., Xu, L., & Zhao, S. (2015). *The Internet of Things: A Survey*. Information Systems Frontiers.

[17] Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., et al. (2016). *TensorFlow: Large-scale machine learning on heterogeneous distributed systems*. arXiv preprint arXiv:1603.04467.

[18] Banks, A., & Gupta, R. (2014). *MQTT Version 3.1.1*. OASIS Standard.

[19] Orłowski, C., & Adrych, M. (2024). *Model of IoT design decision-making processes in Flow Based Programming systems*. Journal of Information and Telecommunication, 8(3), 315–324.

[20] Montori, F., Liao, K., Jayaraman, P. P., Bononi, L., Sellis, T., & Georgakopoulos, D. (2018). *Classification and annotation of open Internet of Things datastreams*. In International Conference on Web Information Systems Engineering (pp. 209–224). Springer.